

Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

«Информатика»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

Архитектура x86-64 и ассемблер NASM

5-13 вариант

Выполнил:

Студент группы N3151

Агакарян А.А.



Проверил:

Грозов В.А.

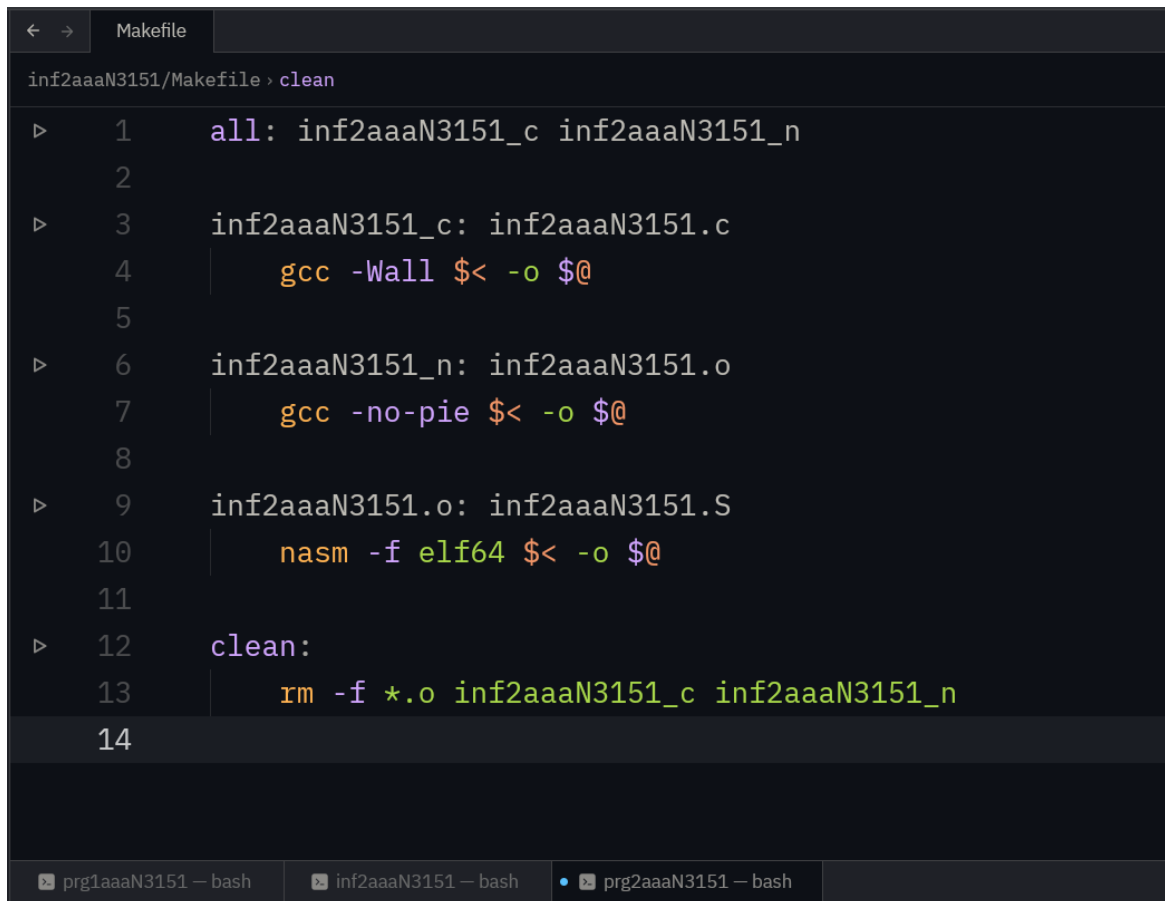
Санкт-Петербург

2025г.

Вариант 5-13

В четных байтах переместить нулевые биты в старшие биты, а в нечетных байтах — в младшие.

Makefile



```
inf2aaaN3151/Makefile > clean

1  all: inf2aaaN3151_c inf2aaaN3151_n
2
3  inf2aaaN3151_c: inf2aaaN3151.c
4      gcc -Wall $< -o $@
5
6  inf2aaaN3151_n: inf2aaaN3151.o
7      gcc -no-pie $< -o $@
8
9  inf2aaaN3151.o: inf2aaaN3151.S
10     nasm -f elf64 $< -o $@
11
12 clean:
13     rm -f *.o inf2aaaN3151_c inf2aaaN3151_n
14
```

примеры работы программ

• inf2aaaN3151 — bash

prg3aaaN3151 — bash

~/Desktop/inf2aaaN3151 via C v13.3.0-gcc

13:06:52 \$./inf2aaaN3151_n

00011011 00000110

11110000 00000011

~/Desktop/inf2aaaN3151 via C v13.3.0-gcc

13:06:58 \$./inf2aaaN3151_n 8888

00100010 10111000

11000000 00001111

~/Desktop/inf2aaaN3151 via C v13.3.0-gcc

13:07:08 \$./inf2aaaN3151_n 21845

01010101 01010101

11110000 00001111

• inf2aaaN3151 — bash

prg3aaaN3151 — bash

~/Desktop/inf2aaaN3151 via C v13.3.0-gcc

13:21:03 \$./inf2aaaN3151_n

11000001 00000001

11100000 00000001

~/Desktop/inf2aaaN3151 via C v13.3.0-gcc

13:21:49 \$./inf2aaaN3151_n

01000111 01010101

11110000 00001111

~/Desktop/inf2aaaN3151 via C v13.3.0-gcc

13:21:51 \$./inf2aaaN3151_n qwerty

Ошибка: 'qwerty' не является числом.

```
inf2aaaN3151 — bash    prg3aaaN3151 — bash

~/Desktop/inf2aaaN3151 via C v13.3.0-gcc
13:22:01 $ ./inf2aaaN3151_n 17823
01000101 10011111
11100000 00111111

~/Desktop/inf2aaaN3151 via C v13.3.0-gcc
14:55:23 $ ./inf2aaaN3151_n 1111
00000100 01010111
10000000 00011111

~/Desktop/inf2aaaN3151 via C v13.3.0-gcc
14:55:29 $ ./inf2aaaN3151_n 41002
10100000 00101010
11000000 00000111
```

дизассемблерный листинг

```
← →  prg3aaaN3151.c  inf2aaaN3151.S  compiled.s  Makefile
prg1aaaN3151/compiled.s
67  move_zeros_to_high:
68  .LFB41:
69      .cfi_startproc
70      endbr64
71      movzx  edi, dil
72      xor edx, edx
73      xor ecx, ecx
74      .p2align 4,,10
75      .p2align 3
76  .L10:
77      mov eax, edi
78      sar eax, cl
79      and eax, 1
80      cmp eax, 1
81      adc edx, 0
82      add ecx, 1
83      cmp ecx, 8
84      jne .L10
85      mov eax, ecx
86      sub eax, edx
87      test  eax, eax
88      jle .L13
89      xor ecx, ecx
90      xor edx, edx
```

prg1aaaN3151/compiled.s

```
91      mov edi, 1
92      .p2align 4,,10
93      .p2align 3
94  .L12:
95      mov esi, edi
96      sal esi, cl
97      add ecx, 1
98      or  edx, esi
99      cmp eax, ecx
100     jne .L12
101     mov eax, edx
102     ret
103     .p2align 4,,10
104     .p2align 3
105  .L13:
106     xor edx, edx
107     mov eax, edx
108     ret
109     .cfi_endproc
```

← →	prg3aaaN3151.c	inf2aaaN3151.S	compiled.s	Makefile
-----	----------------	----------------	------------	----------

prg1aaaN3151/compiled.s

```
115     move_zeros_to_low:
116     .LFB42:
117         .cfi_startproc
118         endbr64
119         movzx    edi, dil
120         xor     edx, edx
121         xor     ecx, ecx
122         .p2align 4,,10
123         .p2align 3
124     .L18:
125         mov     eax, edi
126         sar     eax, cl
127         and     eax, 1
128         cmp     eax, 1
129         adc     edx, 0
130         add     ecx, 1
131         cmp     ecx, 8
132         jne     .L18
133         cmp     edx, 8
134         jge     .L21
135         mov     ecx, 7
136         xor     eax, eax
137         mov     edi, 1
138         .p2align 4,,10
139         .p2align 3
```

← →	prg3aaaN3151.c	inf2aaaN3151.S	compiled.s	Makefile
-----	----------------	----------------	------------	----------

prg1aaaN3151/compiled.s

```
140     .L20:
141         mov     esi, edi
142         sal     esi, cl
143         or      eax, esi
144         mov     esi, ecx
145         sub     ecx, 1
146         cmp     edx, esi
147         jne     .L20
148         ret
149         .p2align 4,,10
150         .p2align 3
151     .L21:
152         xor     eax, eax
153         ret
154         .cfi_endproc
```

ВЫВОД

Когда сравниваешь ручной код на ASM и код, который сгенерировал компилятор, видно несколько вещей. Ручной код более компактный и читаемый: сразу понятно, что делает каждая функция.

Например, `move_zeros_to_high` считает количество единиц и строит маску, чтобы нули оказались в старших битах. В `move_zeros_to_low` почти то же самое, только нули перемещаются в младшие биты. Наш код понятен и можно легко проследить логику.

Код компилятора длиннее и сложнее для чтения. Там много инструкций проверок стека и выравнивания. Логика та же, но компилятор добавляет много деталей, которые нужны для надежности и безопасности, но усложняют чтение.

В целом, результат одинаковый: функции делают то, что нужно. Разница только в том, что ручной код проще для анализа и учебы, а код компилятора защищен и полностью рабочий, но тяжелее для понимания.

Исходный текст програмы с комментариями

`default rel`

`global main`

`extern putchar`

`extern scanf`

`extern srand`

`extern rand`

`extern time`

`extern fprintf`

`extern stderr`

`section .data`

```
fmt_sscanf db "%lli", 0
```

```
fmt_err db "Ошибка: '%s' не является числом.", 10, 0
```

```
section .bss
```

```
value resw 1
```

```
tmp resq 1
```

```
arg1 resq 1
```

```
section .text
```

```
; rdi = value
```

```
print_binary_uint16:
```

```
    push rbx
```

```
    push rcx
```

```
    mov bx, di
```

```
    mov ecx, 15
```

```
.pb_loop:
```

```
    mov ax, bx
```

```
    shr ax, cl
```

```
    and al, 1
```

```
    add al, '0'
```

```
    push rcx
```

```
mov dil, al
call putchar
pop rcx
```

```
cmp ecx, 8
jne .no_space
push rcx
mov dil, ' '
call putchar
pop rcx
```

```
.no_space:
dec ecx
jns .pb_loop
```

```
push rcx
mov dil, 10
call putchar
pop rcx
```

```
pop rcx
pop rbx
ret
```

; dil = byte

; eax = ones

popcount8:

 xor eax, eax

 mov ecx, 8

.pc:

 test dil, 1

 jz .skip

 inc eax

.skip:

 shr dil, 1

 loop .pc

 ret

move_zeros_to_high:

 call popcount8

 mov ecx, eax

 mov eax, 1

 shl eax, cl

 dec eax

 ret

move_zeros_to_low:

 call popcount8

mov ecx, eax

mov eax, 1

shl eax, cl

dec eax

mov edx, 8

sub edx, ecx

mov cl, dl

shl eax, cl

ret

main:

push rbp

mov rbp, rsp

cmp edi, 2

jl .rand_value

; argv[1]

mov rax, [rsi+8]

mov [arg1], rax

mov rdi, rax

mov rsi, fmt_sscanf

mov rdx, tmp

```
xor eax, eax
```

```
call scanf
```

```
cmp eax, 1
```

```
jne .error
```

```
mov ax, [tmp]
```

```
mov [value], ax
```

```
jmp .process
```

```
.rand_value:
```

```
xor edi, edi
```

```
call time
```

```
mov edi, eax
```

```
call srand
```

```
call rand
```

```
mov [value], ax
```

```
.process:
```

```
movzx edi, word [value]
```

```
call print_binary_uint16
```

```
mov al, [value]
```

```
mov dil, al
```

```
call move_zeros_to_high
```

mov bl, al

mov al, [value+1]

mov dil, al

call move_zeros_to_low

mov bh, al

mov ax, bx

mov [value], ax

movzx edi, word [value]

call print_binary_uint16

xor eax, eax

leave

ret

.error:

mov rdi, [rel stderr]

mov rsi, fmt_err

mov rdx, [arg1]

xor eax, eax

call fprintf

mov eax, 1

leave

ret